

# PetConnect

## Documento de Arquitetura e Decisões Técnicas

Este documento consolida as diretrizes técnicas e estruturais definidas para o desenvolvimento do aplicativo PetConnect, um sistema multiplataforma voltado para a centralização do processo de adoção de animais.

### 1. Arquitetura de Software

A aplicação será estruturada sob o modelo de **Arquitetura em Camadas (Layered Architecture)**. Essa escolha promove a separação clara de responsabilidades, fundamental para a organização do TCC. O projeto será dividido em:

- Apresentação (View/UI):** Responsável pela interface gráfica desenvolvida em Python com o framework Flet. Trata a entrada do usuário e exibe os dados (mobile e web).
- Domínio (Business Logic):** Contém as regras de negócio da plataforma, validando ações como os cadastros de animais e os filtros de busca.
- Infraestrutura (Data Access):** Camada encarregada da comunicação externa, gerenciando a conexão direta com o banco de dados PostgreSQL via Supabase.

### 2. Padrões de Projeto (Design Patterns)

O **Repository Pattern (Padrão de Repositório)** será o pilar da comunicação com o banco de dados. A principal vantagem deste padrão é isolar completamente o código que manipula o banco (as requisições SQL/Supabase) do restante do aplicativo, deixando o código das telas limpo.

**Nota sobre o Escopo:** Para garantir a viabilidade e entrega do projeto dentro do cronograma, a funcionalidade de chat interno foi reavaliada e retirada do Produto Mínimo Viável (MVP). A comunicação primária entre adotante e doador poderá ser feita via link direto para WhatsApp. Isso simplifica a complexidade, permitindo foco total na qualidade do fluxo de cadastro e listagem.

### 3. Gerenciamento de Estado da Interface

A reatividade da interface de usuário será controlada pela **Atualização Nativa do Flet** (através do método `page.update()`).

Essa abordagem garante altíssima agilidade de desenvolvimento. Evitando o excesso de complexidade de frameworks de estado baseados em classes logo no início, a prototipação das telas será muito mais veloz, garantindo uma entrega rápida das interfaces prontas.

## 4. Segurança e Autenticação

A proteção da aplicação e a validação dos usuários utilizarão **JWT (JSON Web Tokens) Nativo**, integrado diretamente aos serviços do Supabase.

Estrategicamente, o desenvolvimento será faseado:

- **Fase 1 (Funcionalidades Essenciais):** Construção das telas, navegação e operações de cadastro/busca operando de forma aberta para testes.
- **Fase 2 (Integração da Segurança):** Aplicação da camada de bloqueio e *login*, implementando o armazenamento do token JWT para diferenciar usuários comuns de doadores/ONGs e validar as regras de permissão.